



**FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI**

**KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY**

**Semestrální práce z předmětu
Programování v jazyce C**

Celočíselná kalkulačka s neomezenou přesností

Jméno: Tomáš Svoboda
Datum odevzdání: 31. prosince 2025

Obsah

1	Zadání	2
1.1	Klíčové požadavky	2
2	Analýza úlohy	3
2.1	Reprezentace velkých čísel (BigNum)	3
2.1.1	Volba datové struktury	3
2.1.2	Volba číselné soustavy (Báze)	4
2.2	Zpracování výrazů (Parsování)	4
2.3	Problém se zápornými čísly (Doplňkový kód)	5
3	Popis implementace	6
3.1	Architektura programu	6
3.2	Datové struktury	7
3.2.1	BigNum	7
3.2.2	Token – prvek výrazu	7
3.2.3	Stack – generický zásobník	8
3.3	Zpracování výrazu (Parser)	8
3.3.1	1. Tokenizace	8
3.3.2	2. Shunting-yard (Převod na RPN)	8
3.3.3	3. Vyhodnocení RPN	9
3.4	Implementace algoritmů	10
3.4.1	Sčítání a odčítání	10
3.4.2	Násobení	10
3.4.3	Dělení a modulo	11
3.4.4	Umocňování	12
3.4.5	Faktoriál	12
3.4.6	Negace	12
4	Uživatelská příručka	13
4.1	Překlad	13
4.2	Použití	13
4.2.1	Interaktivní	13
4.2.2	Ze souboru	13
4.3	Formáty čísel	14
4.4	Příkazy	14
4.5	Operátory	14
4.6	Ošetření chybových stavů	15
5	Testování a limity aplikace	15
5.1	Technické limity	15
5.2	Výkonnostní testy	16
6	Závěr	16
6.1	Možná vylepšení	17

1 Zadání

Cílem této semestrální práce je návrh a implementace přenositelné konzolové aplikace v jazyce ANSI C, která slouží jako interpret aritmetických výrazů zapsaných v infixové formě. Vstupem a výstupem budou pouze celá čísla.

1.1 Klíčové požadavky

Na základě zadání byly definovány následující klíčové funkcionality:

1. **Neomezená přesnost:** Program musí být schopen provádět operace nad čísly omezenými pouze pamětí RAM.
2. **Podpora číselných soustav:** Vstup i výstup aplikace musí podporovat zápis čísel v následujících soustavách:
 - Desítková soustava.
 - Binární soustava (zápis s prefixem `0b`).
 - Šestnáctková soustava (zápis s prefixem `0x`).
3. **Reprezentace záporných čísel:** Pro kódování operandů zapsaných v binární a šestnáctkové soustavě musí být použit výhradně dvojkový doplňkový kód.
4. **Sada operátorů:** Interpret musí respektovat standardní matematickou prioritu a asociativitu operátorů. Povinná sada podporovaných operací zahrnuje:
 - **Aritmetické operace:** sčítání (+), odčítání (−), násobení (*), celočíselné dělení (/), zbytek po dělení (%) a mocnění (^).
 - **Unární operátory:** faktoriál (!), unární mínus (−).
5. **Režimy ovládání:** Aplikace musí podporovat dva režimy běhu v závislosti na způsobu spuštění:
 - **Interaktivní režim:** Není-li zadán vstupní soubor, program čte příkazy přímo ze standardního vstupu (`stdin`).
 - **Souborový režim:** Je-li zadán argument příkazové řádky, program načte a postupně vyhodnotí výrazy ze specifikovaného souboru.
6. **Chybové stavy:** V případě chyby (např. neexistující vstupní soubor) program vypíše chybové hlášení a ukončí se s návratovou hodnotou `EXIT_FAILURE`. Při úspěšném ukončení vrací `EXIT_SUCCESS`.

2 Analýza úlohy

Při řešení semestrální práce bylo nutné vyřešit dva hlavní problémy. Prvním z nich je reprezentace čísel v paměti počítače, která jsou větší než rozsah běžných proměnných. Druhým úkolem bylo navrhnout mechanismus pro zpracování a vyhodnocení matematického výrazu zadaného uživatelem.

2.1 Reprezentace velkých čísel (BigNum)

Standardní datové typy v jazyce C (jako `int` nebo `long long int`) mají omezenou velikost. Protože zadání vyžaduje neomezenou přesnost, nebylo možné tyto typy použít a bylo nutné navrhnout vlastní způsob ukládání čísel.

2.1.1 Volba datové struktury

Pro uložení velkého čísla byly zvažovány dvě základní možnosti: spojový seznam a dynamické pole.

1. **Spojový seznam:** Každý bajt (v bázi 256) by byl uložen v samostatném prvku seznamu.
 - *Výhoda:* Snadno se přidávají další bajty, není nutné kopírovat data při zvětšování čísla.
 - *Nevýhoda:* Vysoká paměťová režie (každý bajt vyžaduje ukazatel na další prvek). Především je však komplikované s takovým číslem provádět aritmetické operace – například při násobení je nutné přistupovat k bajtům na různých pozicích, což je u seznamu pomalé a implementačně náročné.
2. **Dynamické pole:** Číslo je uloženo jako souvislý blok paměti spravovaný funkcemi `malloc` a `realloc`.
 - *Výhoda:* Funguje stejně jako běžná pole v C, což výrazně zjednodušuje implementaci cyklů pro sčítání nebo násobení.
 - *Nevýhoda:* Při vyčerpání kapacity je nutné pole zvětšit a data zkopírovat.

Rozhodnutí: Bylo zvoleno **dynamické pole**. Práce s poli je v jazyce C přirozenější a algoritmy pro sčítání či dělení se nad polem implementují mnohem snáze než nad seznamem.

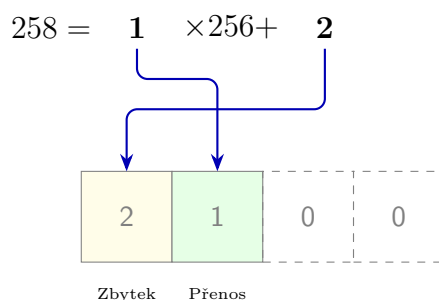
2.1.2 Volba číselné soustavy (Báze)

Dále bylo nutné rozhodnout, v jaké soustavě budou čísla interně reprezentována.

- **Báze 10:** Každý prvek pole by nesl jednu desítkovou číslici (0–9). Výhodou by byl snadný výpis, nevýhodou plýtvání pamětí. Typ `unsigned char` má rozsah 0–255, takže při uložení pouze hodnoty do 9 by většina bitů zůstala nevyužitá.
- **Báze 256:** Využije se celý rozsah jednoho bajtu (typu `unsigned char`). Každý bajt v bázi 256 může nabývat hodnoty 0 až 255.

Rozhodnutí: Byla zvolena **báze 256**. Jedná se o nejefektivnější využití paměti, protože jeden prvek pole přesně odpovídá jednomu bajtu. Zároveň tato volba zjednodušuje práci s bitovými operacemi, které jsou nezbytné pro převody do binární soustavy.

Pro lepší představu uvádím konkrétní příklad uložení čísla ve struktuře `BigNum`. Mějme číslo **258**. Protože používám bázi 256, číslo se dělí: $258 = 1 \times 256 + 2$. Do pole se tyto hodnoty uloží v pořadí od nejmenšího řádu (Little-Endian).

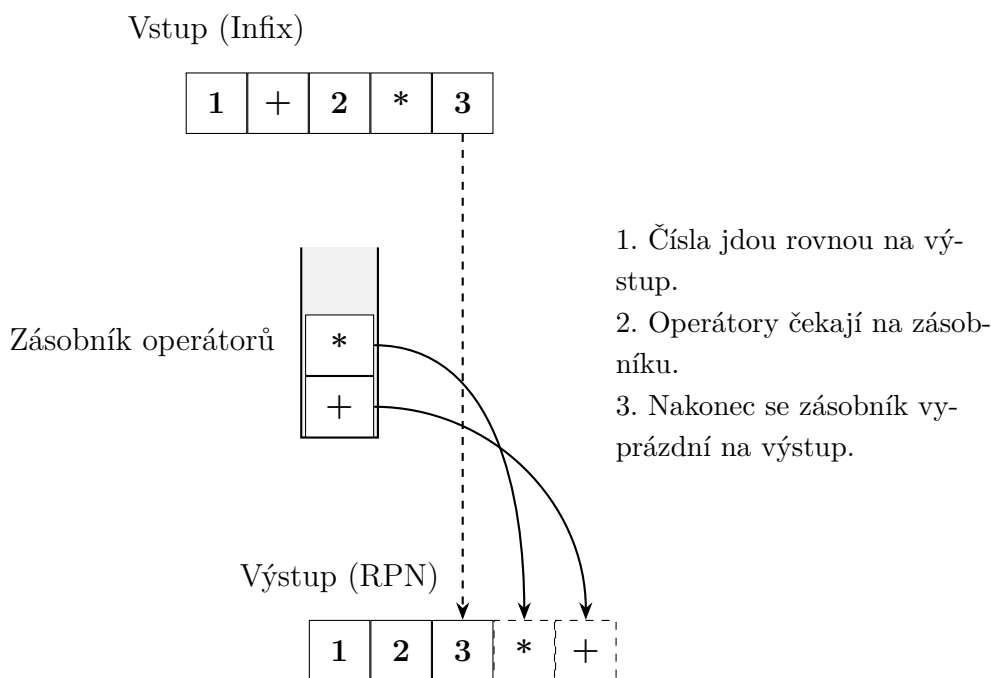


Obrázek 1: Ukázka uložení čísla 258 v poli. Hodnoty se ukládají od nejnižšího řádu.

2.2 Zpracování výrazů (Parsování)

Dalším úkolem bylo zajistit vyhodnocování výrazů typu $1 + 2 * (3 - 4)$. Problém spočívá v tom, že počítač čte text zleva doprava, ale v matematice má násobení přednost před sčítáním a závorky mění pořadí operací.

Pro řešení tohoto problému byl zvolen algoritmus **Shunting-yard**, který přeuspořádá čísla a operátory tak, aby vznikl zápis v Reverzní polské notaci (RPN).



Obrázek 2: Princip algoritmu Shunting-yard: převod z $1+2*3$ na $1\ 2\ 3\ *\ +$.

- **Proč RPN?** V tomto zápisu nejsou potřeba závorky a operátory jsou přesně tam, kde mají být.
- **Výpočet:** Výraz v RPN se pak dá velmi snadno spočítat pomocí jednoho zásobníku. Čtu čísla, dávám je na zásobník, a když přijde operátor, vezmu poslední dvě čísla, spočítám výsledek a vrátím ho zpět.

2.3 Problém se zápornými čísly (Doplňkový kód)

Zadání vyžaduje, aby se binární a šestnáctková čísla zadávala a vypisovala ve dvojkovém doplňkovém kódu. Toto však neodpovídá způsobu, jakým jsou čísla uložena uvnitř programu. Struktura `BigNum` uchovává velikost čísla a znaménko odděleně (pomocí příznaku `is_negative`), což je výhodnější pro operace dělení či násobení.

Bylo tedy nutné implementovat převody při vstupu a výstupu:

- **Načítání:** Pokud uživatel zadá binární číslo, jehož nejvyšší bit je roven 1, jedná se o záporné číslo v doplňkovém kódu. Je nutné provést inverzi bitů a přičíst jedničku, čímž se získá absolutní hodnota čísla. Ta se uloží do struktury spolu s příznakem zápornosti.
- **Výpis:** Při výpisu záporného čísla nelze pouze vypsát jeho binární reprezentaci. Je třeba provést převod zpět do doplňkového kódu (negace a přičtení 1) a teprve tento výsledek zobrazit.

Určení šířky doplňkového kódu: Šířka (počet bitů) doplňkového kódu je odvozena přímo od délky vstupního řetězce. Nejvyšší zadaný bit je považován za znaménkový. Například řetězec `0b111` má 3 bity, z nichž první (nejvyšší) je 1, což indikuje záporné číslo. Po převodu z doplňkového kódu tedy $0b111 = -1$.

3 Popis implementace

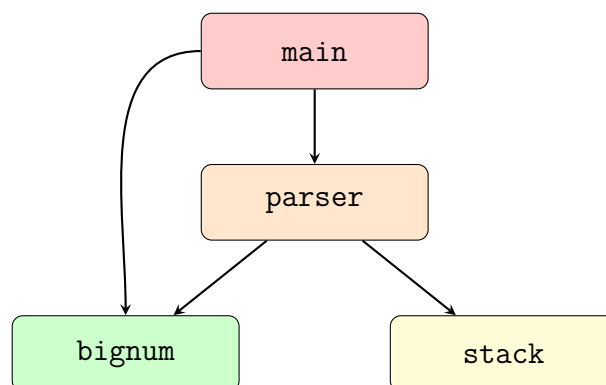
Tato kapitola popisuje vnitřní strukturu aplikace. Zaměřil jsem se na rozdělení kódu do modulů, návrh datových struktur pro velká čísla a vysvětlení algoritmů, které kalkulačka používá.

3.1 Architektura programu

Z důvodu přehlednosti byl program rozdělen do čtyř hlavních modulů, z nichž každý má jasně definovanou odpovědnost:

- **main** – Vstupní bod programu. Řídí komunikaci s uživatelem, načítá vstup (z konzole nebo souboru) a předává ho ke zpracování.
- **parser** – Modul pro analýzu textu. Rozdělí vstup na jednotlivé tokeny, zkontroluje syntaxi a převede výraz do postfixové notace.
- **bignum** – Jádro kalkulačky. Obsahuje strukturu pro velká čísla a implementaci všech matematických operací.
- **stack** – Generický zásobník využívaný na více místech (pro operátory v parseru i pro čísla při výpočtu).

Vzájemné vazby ukazuje následující schéma. Hlavní modul **main** volá **parser** pro vyhodnocení výrazů a také přímo využívá **bignum** pro správu výsledků (inicializace, uvolnění paměti, převod na řetězec). Modul **parser** pak používá **bignum** pro výpočty a **stack** pro dočasné ukládání dat.



Obrázek 3: Závislosti mezi moduly programu.

3.2 Datové struktury

Klíčem k funkční kalkulačce bylo vymyslet, jak efektivně ukládat data v paměti.

3.2.1 BigInt

Hlavní struktura pro velká čísla. Je to dynamické pole bajtů, kde index 0 je nejnižší řád.

```
1 struct BigInt {
2     unsigned char *bytes; /* Dynamicky alokované pole bajtu */
3     size_t capacity;      /* Počet alokovaných bajtů */
4     size_t size;          /* Počet skutečně použitých bajtů */
5     int is_negative;      /* 1 pokud je záporné, jinak 0 */
6 };
```

Listing 1: BigInt

3.2.2 Token – prvek výrazu

Při čtení vstupu je text rozdělen na tokeny. Token může být číslo, operátor nebo závorka. Pro úsporu paměti je použito sjednocení (`union`) – jeden token obsahuje buď číslo, nebo operátor, nikdy ne obojí současně.

```
1 struct Token {
2     enum TokenType type; /* Typ: číslo, operátor, závorka */
3     union {
4         struct BigInt number; /* Hodnota, pokud je to číslo */
5         struct Operator op;   /* Vlastnosti, pokud je to
6                               operátor */
7     } value;
8 };
```

Listing 2: Token

3.2.3 Stack – generický zásobník

Místo samostatných zásobníků pro čísla a operátory byl implementován jeden generický zásobník, který dokáže ukládat prvky libovolné velikosti. Díky parametru `item_size` je použitelný pro libovolný datový typ.

```
1 struct stack {
2     size_t capacity;    /* Maximalni pocet prvku */
3     size_t item_size;   /* Velikost jednoho prvku v bajtech */
4     size_t sp;          /* Stack pointer (index vrcholu) */
5     void *items;        /* Pole pro ulozeni dat */
6 };
```

Listing 3: Stack

3.3 Zpracování výrazu (Parser)

Tato část programu zajišťuje zpracování textového vstupu (např. "1 + 2 * 3") a jeho vyhodnocení. Proces probíhá ve třech krocích:

3.3.1 1. Tokenizace

Vstupní řetězec je nejprve rozdělen na pole tokenů. Program čte znak po znaku:

- Při detekci číslice je načteno celé číslo (včetně prefixů 0x a 0b) a převedeno do struktury `BigNum`.
- Při detekci operátoru nebo závorky je vytvořen příslušný token.

Detekce unárního mínus: Znak `-` může znamenat buď odčítání (binární), nebo negaci čísla (unární). Parser rozpozná unární mínus, pokud:

1. Je mínus na úplném začátku výrazu (např. `-5`).
2. Následuje bezprostředně po levé závorce (např. `(-5)`).
3. Následuje po jiném operátoru (např. `3 * -2`). Výjimkou je faktoriál (`!`), který je postfixový, takže za ním následuje běžné odčítání.

Pokud je detekován unární mínus, token se interně označí symbolem `~`, aby se odlišil od odčítání a měl správnou (vyšší) prioritu.

3.3.2 2. Shunting-yard (Převod na RPN)

Pro vyřešení priorit operátorů je využíván Dijkstrův algoritmus **Shunting-yard**. Ten převede infixový zápis na postfixový (RPN), se kterým se počítači pracuje lépe. Operátory s vyšší prioritou se v něm dostanou na řadu dříve.

Pro správnou funkci algoritmu je nezbytné definovat priority a asociativity všech operátorů. Tyto hodnoty jsou shrnuty v následující tabulce:

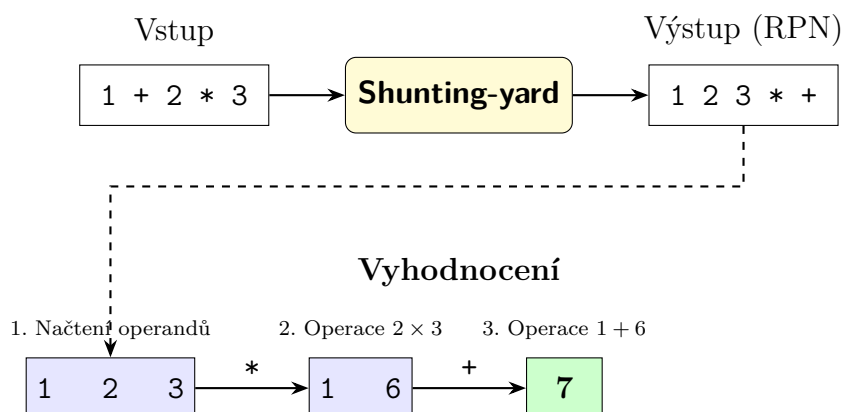
Tabulka 1: Priority a asociativity operátorů

Operátor	Priorita	Asociativita	Typ
!	5	zleva	unární (postfix)
^	4	zprava	binární
~ (unární −)	3	zprava	unární (prefix)
*, /	3	zleva	binární
%	2	zleva	binární
+, −	1	zleva	binární

Vyšší číslo priority znamená, že operátor bude vyhodnocen dříve. Asociativita určuje pořadí vyhodnocení operátorů se stejnou prioritou.

3.3.3 3. Vyhodnocení RPN

Výraz v RPN (např. 1 2 3 * +) se počítá jednoduše zleva doprava pomocí zásobníku čísel. Celý postup ukazuje schéma níže.



Obrázek 4: Zpracování výrazu

3.4 Implementace algoritmů

V této části popíšu, jak program uvnitř počítá. U základních operací jsem se inspiroval tím, jak počítáme my lidé na papíře (pod sebou), u těch složitějších jsem využil efektivnější algoritmy, aby výpočet netrval věčnost.

3.4.1 Sčítání a odčítání

Sčítání funguje na principu klasického sčítání pod sebe s přenosem (carry), jen místo s desítkovými číslicemi pracuji s bajty (0–255). Algoritmus prochází čísla od nejnižšího bajtu k nejvyššímu. Pokud součet na dané pozici přesáhne 255, zapíšu zbytek a jedničku přenesu do vyššího řádu.

U odčítání je postup podobný, jen si místo přenosu "půjčuji" z vyššího řádu. Než ale začnu počítat, vyřeším znaménka:

- Pokud odčítám záporné číslo ($a - (-b)$), převedu to na sčítání ($a + |b|$).
- Pokud bych měl odečíst větší číslo od menšího, čísla prohodím a výsledku nastavím záporné znaménko.

3.4.2 Násobení

Pro násobení jsem zvolil algoritmus „školního násobení“. Je to přesně ten postup, který se učí na základní škole – každou číslicí (zde bajtem) jednoho čísla vynásobím celé druhé číslo a výsledky s posunem sečtu.

```
1 ALGORITMUS Nasobeni(A, B):
2   Vytvor vysledek o velikosti size(A) + size(B)
3   Nuluj vysledek
4
5   PRO i OD 0 DO size(A):      (prochazime prvni cislo)
6     carry = 0
7     PRO j OD 0 DO size(B):    (prochazime druhe cislo)
8       // Vypocet soucinu bajtu + pripocteni k aktualni pozici
9       soucin = A[i] * B[j] + vysledek[i+j] + carry
10
11     vysledek[i+j] = soucin MOD 256
12     carry = soucin / 256
13     POKUD carry > 0:
14       vysledek[i+j+1] += carry
15
16   Nastav vysledne znamenko
17   Odstran vedouci nuly
18   VRAT vysledek
```

Listing 4: Pseudokód algoritmu násobení

3.4.3 Dělení a modulo

Dělení bylo implementačně nejnáročnější. Protože dělit velká čísla bajt po bajtu je složité, použil jsem metodu **dělení po bitech** (Bitwise Long Division). Funguje to podobně jako písemné dělení, ale ve dvojkové soustavě:

1. Procházím dělenec bit po bitu od nejvyššího k nejnižšímu.
2. V každém kroku posunu aktuální zbytek doleva a "stáhnou" další bit dělence.
3. Pokud se dělitel vejde do zbytku, odečtu ho a do výsledku (podílu) zapíšu jedničku.

```
1 ALGORITMUS Deleni(A, B):
2   POKUD B == 0: CHYBA // Deleni nulou
3
4   R = 0 // Zbytek
5   Q = 0 // Podil
6
7   PRO kazdy bajt i od nejvyssiho po nejnizsi:
8     PRO kazdy bit k od 7 do 0:
9       R = R << 1
10      Pridej bit A[i][k] do R
11
12      POKUD R >= |B|:
13        R = R - |B|
14        Nastav bit Q[i][k] na 1
15
16   Urceni znamenska Q a R
17   VRAT Q, R
```

Listing 5: Pseudokód algoritmu dělení

Operace modulo vrací zbytek po dělení.

3.4.4 Umocňování

Kdybych měl počítat A^{1000} tak, že tisíckrát vynásobím A , trvalo by to dlouho. Proto jsem implementoval **binární umocňování** (Square and Multiply). Tento trik využívá toho, že exponent můžeme rozepsat ve dvojkové soustavě. Místo násobení po jedné číslo raději mocním na druhou, což snižuje počet kroků.

```
1 ALGORITMUS Mocnina(base, exp):
2   POKUD exp < 0: CHYBA (Zaporný exponent nepodporovan)
3   POKUD exp == 0: VRAT 1
4
5   result = 1
6
7   DOKUD exp > 0:
8     POKUD je exp liche:
9       result = result * base
10
11     exp = exp / 2
12
13     POKUD exp > 0:
14       base = base * base
15
16   VRAT result
```

Listing 6: Pseudokód algoritmu umocňování

3.4.5 Faktoriál

Jednoduchý cyklus – začnu od 1 a násobím čísla 2, 3, 4... až do n .

3.4.6 Negace

Změna příznaku `is_negative`. Nula zůstává vždy kladná.

4 Uživatelská příručka

4.1 Překlad

Program je napsaný ve standardním ANSI C, takže k jeho přeložení nejsou potřeba žádné speciální knihovny.

Na Linuxu:

```
$ make
```

Vytvoří se `calc`.

Na Windows:

```
> make -f Makefile.win
```

Vytvoří se `calc.exe`.

4.2 Použití

Program má dva režimy.

4.2.1 Interaktivní

Když ho spustíte bez parametrů, můžete psát příklady. Ukončíte ho příkazem `quit`.

```
$ ./calc
> 10 + 20
30
> 0b101 * 2
10
> quit
```

4.2.2 Ze souboru

Když dáte jako parametr soubor, načte příklady z něj.

```
$ ./calc test_input.txt
> 10 + 20
30
> hex
hex
> 255 + 1
0x100
```

4.3 Formáty čísel

- Desítková - normálně (123, -456)
- Binární - s prefixem 0b (0b110)
- Hex - s prefixem 0x (0xFF)

4.4 Příkazy

- `dec` - výstup v desítkové soustavě (výchozí)
- `bin` - výstup v binární soustavě
- `hex` - výstup v hexadecimální soustavě
- `out` - jaká soustava je nastavená
- `quit` - konec

4.5 Operátory

Tabulka 2: Operátory

+	sčítání
-	odčítání
*	násobení
/	dělení
%	modulo
^	mocnina
!	faktoriál

4.6 Ošetření chybových stavů

Program korektně reaguje na neplatné vstupy a chybové situace:

Tabulka 3: Přehled chybových stavů a jejich hlášení

Situace	Chybová hláška
Dělení nulou	Division by zero!
Syntaktická chyba (chybný výraz)	Syntax error!
Faktoriál záporného čísla	Input of factorial must not be negative!
Záporný exponent	Negative exponent is not allowed.
Nedostatek paměti	Out of memory.
Neplatný příkaz	Invalid command "..."

5 Testování a limity aplikace

Pro ověření stability a robustnosti programu byla provedena série testů.

5.1 Technické limity

Při návrhu programu byla zavedena některá omezení, která jsou shrnuta v následující tabulce.

Tabulka 4: Přehled technických limitů kalkulačky

Omezení	Hodnota	Vysvětlení
Vstupní řádek	1024 znaků	Konstanta <code>MAX_LINE_LENGTH</code> v <code>main.c</code> . Delší výraz na jednom řádku program nenačte celý.
Velikost čísla	Omezeno RAM	Struktura <code>BigNum</code> se dynamicky zvětšuje, takže limit je jen volná operační paměť.
Výstupní soustava	Dec/Bin/Hex	Jiné soustavy (např. osmičková) nejsou implementovány.

5.2 Výkonnostní testy

Pro zjištění výkonnostních charakteristik byly prováděny výpočty s velkými čísly (faktoriály a mocniny). Měření probíhalo na stolním počítači s operačním systémem Linux.

Tabulka 5: Naměřené časy výpočtů

Příklad	Počet cifer výsledku	Čas výpočtu
100!	158	< 0.1 s
1 000!	2 568	< 0.1 s
10 000!	35 600	cca 1 s
20 000!	77 300	6.5 s
50 000!	213 000	46 s
100 000!	460 000	cca 3 min
$2^{10\,000}$	3 000	< 0.1 s
$2^{100\,000}$	30 000	< 0.1 s
$2^{1\,000\,000}$	301 000	78 s

6 Závěr

Cílem této semestrální práce bylo vytvořit kalkulačku, která není omezena velikostí standardních datových typů. Lze konstatovat, že toto zadání bylo úspěšně splněno. Výsledná aplikace zvládá všechny požadované operace a dokáže v reálném čase zpracovávat výrazy s obrovskými čísly.

Z naměřených testů (uvedených v předchozí kapitole) lze vyvodit několik důležitých závěrů o chování programu:

1. **Stabilita paměti:** Program úspěšně spočítal $2^{1\,000\,000}$, což je číslo s více než 300 000 ciframi. To potvrzuje, že dynamická správa paměti funguje správně a nedochází k únikům paměti ani při extrémních hodnotách.
2. **Limity rychlosti:** Zatímco do cca 30 000 cifer je výpočet prakticky okamžitý, u větších čísel se projevuje nevýhoda použitého algoritmu násobení se složitostí $O(N^2)$.

6.1 Možná vylepšení

Ačkoliv je program plně funkční, existuje prostor pro další optimalizaci a rozšíření:

1. **Optimalizace násobení:** Pro extrémně velká čísla by bylo vhodné implementovat algoritmus **Karatsuba** (složitost $O(N^{1.585})$), což by výrazně urychlilo výpočty.
2. **Podpora dalších soustav:** Rozšíření parseru o osmičkovou soustavu nebo možnost zadání obecné báze.
3. **Záporné exponenty:** Doplnění podpory pro racionální čísla nebo plovoucí řádovou čárku.
4. **Rozšířená sada operací:** Přidání funkcí jako odmocnina, logaritmus apod.

Reference

- [1] WIKIPEDIA CONTRIBUTORS. *Shunting-yard algorithm* [online]. 2025 [cit. 22.11.2025].
Dostupné z: https://en.wikipedia.org/wiki/Shunting-yard_algorithm
- [2] LITERATEPROGRAMS CONTRIBUTORS. *Shunting yard algorithm (C)* [online]. [b.r.] [cit. 22.11.2025].
Dostupné z: https://www.literateprograms.org/shunting_yard_algorithm__c_.html
- [3] WIKIPEDIA CONTRIBUTORS. *Reverse Polish notation* [online]. 2025 [cit. 22.11.2025].
Dostupné z: https://en.wikipedia.org/wiki/Reverse_Polish_notation
- [4] WIKIPEDIA CONTRIBUTORS. *Division algorithm* [online]. 2025 [cit. 22.11.2025].
Dostupné z: https://en.wikipedia.org/wiki/Division_algorithm
- [5] WIKIPEDIA CONTRIBUTORS. *Exponentiation by squaring* [online]. 2025 [cit. 22.11.2025].
Dostupné z: https://en.wikipedia.org/wiki/Exponentiation_by_squaring
- [6] FINLEY, Thomas. *Two's Complement* [online]. Cornell University, 2000 [cit. 22.11.2025].
Dostupné z: <https://cs.cornell.edu/~tomf/notes/cps104/twoscomp.html>
- [7] PÁRTL, František. *Podklady ke cvičení KIV/PC: Implementace zásobníku a parseru* [zdrojový kód]. Západočeská univerzita v Plzni, [b.r.] [cit. 22.11.2025].